Spring Security – Complete Mental Model (From Zero to JWT)

1️⃣ Why Do We Need Spring Security?

Without security, any client can:

- Call your APIs
- Access private data
- Modify resources
- Impersonate users

Spring Security solves:

- **Authentication** → Who are you?
- **Authorization** → What are you allowed to do?
- **Protection** → CSRF, session fixation, password hashing

2️⃣ Core Idea of Spring Security (One Line)

> **Every HTTP request passes through a chain of filters before it reaches your controller.**

These filters decide:

- Is the user authenticated?
- Is the user authorized?

3️⃣ What Is a Spring Security Filter?

A filter is a **middleware** (just like Express middleware).

Core method

```
doFilter(request, response, chain)
```

Equivalent to:

```
next()
```

Responsibilities

- Read request headers
- Authenticate user (if needed)
- Set SecurityContext
- Pass request forward

4️⃣ Spring Security Filter Chain (High Level)

```
Request
  ↓
SecurityContextPersistenceFilter
  ↓
JwtFilter (custom)
  ↓
UsernamePasswordAuthenticationFilter
  ↓
AuthorizationFilter
  ↓
Controller
```

> Filters run **before controllers**

5️⃣ Authentication vs Authorization (Very Important)

Concept	Meaning
Authentication	Proving identity
Authorization	Checking permissions

JWT is **NOT authentication**. JWT is **proof of authentication**.

6️⃣ AuthenticationManager (The Orchestrator)

What it does

```
Takes Authentication object → Finds correct provider → Returns authenticated
user
```

Interface

```
Authentication authenticate(Authentication authentication)
```

AuthenticationManager **does not authenticate itself**. It delegates.

7️⃣ AuthenticationProvider (The Real Authenticator)

Responsibility

> Verify credentials

Examples:

- Username + Password
- LDAP
- OAuth

DaoAuthenticationProvider

Used for:

```
UsernamePasswordAuthenticationToken
```

Checks:

- User exists
- Password matches (BCrypt)
- Account enabled

8️⃣ How Spring Chooses the Correct Provider

Each provider declares:

```
boolean supports(Class<?> authentication)
```

Example:

```
return UsernamePasswordAuthenticationToken.class
       .isAssignableFrom(authentication);
```

Flow

```
AuthenticationManager
  ↓
Loops providers
  ↓
```

```
Calls supports()
  ↓
Uses matching provider
```

9️⃣ User vs UserDetails (Critical Difference)

Your User Entity

```
class User {
  id
  username
  password
}
```

Spring Security UserDetails

```
interface UserDetails {
  getUsername()
  getPassword()
  getAuthorities()
}
```

> Spring Security **never works with your entity directly**

You adapt it using `UserDetailsService`.

🔟 UserDetailsService

Purpose

> Load user for authentication

```
UserDetails loadUserByUsername(String username)
```

Used by:

- DaoAuthenticationProvider
- JWT Filter (for restoring identity)

1️⃣1️⃣ Login Flow (Username + Password)

```
POST /login
  ↓
UsernamePasswordAuthenticationFilter
  ↓
AuthenticationManager
  ↓
DaoAuthenticationProvider
  ↓
User authenticated
  ↓
JWT generated
```

JWT is created **after authentication succeeds**.

1️⃣2️⃣ Why JWT Does NOT Use AuthenticationProvider

JWT:

- Has no password
- Has no credentials
- Has only a signed claim

So:

- Nothing to authenticate
- Only verify signature & expiry

➡️ **Handled by a Filter, not a Provider**

1️⃣3️⃣ JWT Filter (Most Important Piece)

Responsibility

- Intercept request
- Extract token
- Validate token
- Restore authentication

What it DOES

```
SecurityContextHolder.getContext()
    .setAuthentication(auth);
```

What it DOES NOT do

❌Login ❌Password check ❌Authentication decision

1️⃣4️⃣Token Validation – Where It Happens

```
jwtUtil.validateToken(token);
```

Checks:

- Signature
- Expiration
- Issuer

Spring Security **never validates JWT by default**. YOU do it.

1️⃣5️⃣Why We Still Need DaoAuthenticationProvider

Phase	Component
Login	DaoAuthenticationProvider
Requests	JwtFilter

JWT does not replace providers. JWT replaces sessions.

1️⃣6️⃣Stateless vs Stateful Security

Stateful (Sessions)

- Server stores session
- Memory heavy

Stateless (JWT)

- No server memory
- Token carries identity

JWT = Stateless security

1️⃣7️⃣CSRF vs JWT

CSRF	JWT
Browser attack	Header-based

CSRF	JWT
Cookie-based	Authorization header
Needs protection	Naturally safe

JWT APIs usually disable CSRF.

1️⃣8️⃣ Password Encryption

Spring uses:

```
BCryptPasswordEncoder
```

Why BCrypt?

- Salted
- Slow (prevents brute force)

Never store plain passwords.

1️⃣9️⃣ Full Request Lifecycle (JWT)

```
Client Request
  ↓
JwtFilter
  ↓
Token validation
  ↓
SecurityContext populated
  ↓
Authorization check
  ↓
Controller
```

2️⃣0️⃣ What You Should Master Next 🚀

1️⃣ Write your own JWT filter from scratch 2️⃣ Debug filter chain order 3️⃣ Add role-based authorization 4️⃣ Deep dive: CSRF vs JWT 5️⃣ Stateless vs Stateful security trade-offs

---

✅ Final One-Liners to Remember

- **Filters intercept requests**

- **AuthenticationManager delegates**
- **AuthenticationProviders authenticate credentials**
- **JWT restores authentication, it does not authenticate**
- **SecurityContextHolder = current user identity**

This is the COMPLETE Spring Security mental model.

🔁Request-by-Request Execution Diagram (JWT Todo API)

🚫1️⃣Login Request Flow (`POST /auth/login`)

```
Client
  │  (username + password)
  ▼
UsernamePasswordAuthenticationFilter
  │ creates UsernamePasswordAuthenticationToken (UNAUTHENTICATED)
  ▼
AuthenticationManager
  │ delegates
  ▼
DaoAuthenticationProvider
  │ loadUserByUsername()
  │ passwordEncoder.matches()
  ▼
Authentication (AUTHENTICATED)
  │ returned
  ▼
SecurityContextHolder
  │ setAuthentication()
  ▼
JWT generated & returned
```

✔️ **AuthenticationManager is used here** ✔️ **AuthenticationProvider (DAO) validates credentials**

🚫2️⃣Authenticated Request Flow (`POST /todos`)

```
Client
  │ Authorization: Bearer <JWT>
  ▼
JwtFilter (OncePerRequestFilter)
  │ extract token
  │ validate token (signature + expiry)
  │ loadUserByUsername()
  │ create UsernamePasswordAuthenticationToken (AUTHENTICATED)
  │ SecurityContextHolder.setAuthentication()
```

```
  ▼
Spring Security Authorization
  │ checks roles / authentication
  ▼
Controller
```

✔️ **NO AuthenticationManager here** ✔️ **NO AuthenticationProvider here** ✔️ JWT itself is the proof

---

🧰 How Authentication Object Is Created (Even When You Don't)

Case 1: Username + Password

Spring internally does:

```
new UsernamePasswordAuthenticationToken(username, password)
```

This happens inside:

- `UsernamePasswordAuthenticationFilter`

You NEVER write this manually.Let's understand this

Your code (anchor)

```
@PostMapping("/login")
public String login(@RequestBody LoginRequest req) {

    Authentication auth =
        authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                req.getUsername(),
                req.getPassword()
            )
        );

    return jwtUtil.generateToken(auth.getName());
}
```

You're asking:

> "I'm clearly creating `UsernamePasswordAuthenticationToken` myself — so where is Spring creating it internally?"

🔴Key Distinction (VERY IMPORTANT)

You are creating **ONLY the *input* Authentication object**
Spring creates **the *output* Authentication object**

They are **NOT the same object**

Step-by-Step Execution (Internals Exposed)

1️⃣You create an unauthenticated token

```
new UsernamePasswordAuthenticationToken(username, password)
```

Internally this sets:

```
principal = username
credentials = password
authorities = null
authenticated = false
```

This object means:

> "Please authenticate this user"

2️⃣You pass it to AuthenticationManager

```
authenticationManager.authenticate(inputToken);
```

Now Spring takes control.

3️⃣AuthenticationManager loops providers

Internally (`ProviderManager.authenticate()`):

```
for (AuthenticationProvider provider : providers) {
    if (provider.supports(authentication.getClass())) {
        return provider.authenticate(authentication);
    }
}
```

➡️`DaoAuthenticationProvider` supports `UsernamePasswordAuthenticationToken`

4️⃣ DaoAuthenticationProvider does validation

Inside Spring Security (simplified):

```
UserDetails user = userDetailsService.loadUserByUsername(username);

if (!passwordEncoder.matches(rawPassword, user.getPassword())) {
    throw new BadCredentialsException();
}
```

✔️ Username valid
✔️ Password valid

5️⃣✅ SPRING CREATES A NEW TOKEN HERE ✅

Inside `AbstractUserDetailsAuthenticationProvider`:

```
return new UsernamePasswordAuthenticationToken(
    userDetails,            // principal
    null,                   // credentials wiped
    userDetails.getAuthorities()
);
```

This token is:

```
authenticated = true
```

> **THIS is the token Spring creates internally**

6️⃣ AuthenticationManager returns the NEW token

```
Authentication auth = authenticationManager.authenticate(...);
```

⚠️ `auth` ≠ the token you passed in

You passed:

```
(username, password) → unauthenticated
```

Spring returned:

```
(UserDetails, null, authorities) → authenticated
```

Proof: Why password becomes `null`

Try debugging:

```
auth.getCredentials(); // returns null
```

Spring **explicitly clears it** for security.

7️⃣ Why `auth.getName()` works

`getName()` comes from `Principal`:

```
@Override
public String getName() {
    return userDetails.getUsername();
}
```

So JWT gets correct username.

8️⃣ Where This Fits in JWT Flow

Login endpoint:

✔️ AuthenticationManager
✔️ AuthenticationProvider
✔️ PasswordEncoder
✔️ Internal token creation

JWT requests:

❌ AuthenticationManager
❌ AuthenticationProvider
✔️ JwtFilter
✔️ SecurityContext restoration

Final Mental Model (Memorize This)

- You create the **request Authentication**

- Spring creates the **authenticated Authentication**
- AuthenticationManager is a **factory + router**
- AuthenticationProvider does the **real work**

One-Line Summary

> **You never authenticate users — Spring does.**
> **You only ask it to.**

If you want next, I can:

- Show **actual Spring source code**
- Debug with **breakpoints**
- Explain **multiple providers**
- Build **custom AuthenticationProvider**

Just say the word ⚫

but the authentication will be genrated by jwt

Great — this is the **last missing puzzle piece**, and once this clicks, **Spring Security will fully make sense**.

Your statement:

> **"but the authentication will be generated by JWT"**

♓**YES — but only AFTER login**
♓**NO — not during login**

Both are true depending on the phase.

Let's lock this in clearly.

📵There are TWO authentication moments

Spring Security has **two different authentication responsibilities**:

PhaseWho authenticatesHow		
Login	AuthenticationManager	Username + Password
Every request after login	JwtFilter	JWT token

1️⃣ Login Phase (JWT does NOT authenticate)

Request:

```
POST /login
username + password
```

What happens:

```
authenticationManager.authenticate(
   new UsernamePasswordAuthenticationToken(username, password)
);
```

Why JWT is NOT used here:

- JWT does **not exist yet**
- Password must be validated
- Database must be checked

➡️ **Only AuthenticationManager can do this**

Result:

Spring returns an **authenticated Authentication object**
and **you generate JWT from it**

```
jwtUtil.generateToken(auth.getName());
```

⚫ JWT is a **product of authentication**, not the authenticator here.

2️⃣ JWT Phase (JWT DOES authenticate)

Request:

```
GET /todos
Authorization: Bearer <JWT>
```

What happens:

```
JwtFilter.doFilterInternal()
```

Inside the filter:

```
String username = jwtUtil.extractUsername(token);
jwtUtil.validateToken(token);   // signature + expiry

UserDetails user = userDetailsService.loadUserByUsername(username);

Authentication auth =
    new UsernamePasswordAuthenticationToken(
        user, null, user.getAuthorities()
    );

SecurityContextHolder.getContext().setAuthentication(auth);
```

✅**THIS is authentication generated by JWT**

No AuthenticationManager
No AuthenticationProvider
No password check

Because:

> JWT already proves identity

3️⃣ Why Spring accepts JWT authentication

Spring Security **does not care HOW Authentication was created**

It only checks:

```
SecurityContextHolder.getContext().getAuthentication()
```

If:

- not null
- `isAuthenticated() == true`

➡️Request is treated as authenticated

4️⃣ Why JWT bypasses AuthenticationManager

AuthenticationManager is for **credential verification**

JWT is:

- Signed
- Tamper-proof

- Self-contained

So:
✔️Verify token
✔️Restore Authentication
✔️Done

5️⃣Unified Mental Model (✅Remember This)

⚫**AuthenticationManager authenticates credentials**

📳**JWT restores authentication**

SecurityContextHolder is the authority

6️⃣One Diagram to Rule Them All

```
LOGIN
-----
Client → AuthenticationManager
       → AuthenticationProvider
       → PasswordEncoder
       → Authentication (authenticated)
       → JWT generated


EVERY REQUEST
-------------
Client → JwtFilter
       → Validate JWT
       → Create Authentication
       → SecurityContextHolder
       → Controller
```

Final Verdict (Answer to your sentence)

❌"Authentication will be generated by JWT" (always)

✔️Corrected version:

✅**Authentication is generated by AuthenticationManager at login**
Authentication is restored by JWT for every request

If you want, next I can:

- Show **Spring source code**
- Implement **JWT AuthenticationProvider**
- Compare **OAuth2 vs JWT**
- Show **what breaks if you skip filters**

Just tell me ⚫

public TodoEntity create(@RequestBody TodoEntity todo, Authentication auth) { todo.setUsername(auth.getName()); return repo.save(todo); } How does Authentication auth is recieved here and how does it let controller know this is authenticated cause it doesnt go to AuthenticationProvider and dosent return true or false

Excellent question. This is **the exact moment where Spring Security feels "magical"**—but it's actually very deterministic.

Let's break this **slowly, mechanically, and end-to-end** so you see **why this works without AuthenticationProvider**.

⚫Short Answer (then we go deep)

```
public TodoEntity create(@RequestBody TodoEntity todo,
                         Authentication auth)
```

♓ `Authentication auth` **comes from **``
♓It was **already set earlier by a security filter (JwtFilter)**
♓Controller **never talks to AuthenticationProvider**
♓AuthenticationProvider is **only for login**

Core Rule (Memorize This)

> **Controllers NEVER authenticate**
>
> They only **consume authentication that already exists**

Authentication happens **before the controller**.

🔄 Full Request Flow (JWT request)

Request:

```
POST /todos
Authorization: Bearer <JWT>
```

1️⃣ Request enters Spring Security filter chain

Before **ANY controller method runs**, Spring does this:

```
Client
 ↓
SecurityFilterChain
 ↓
JwtFilter
```

2️⃣ JwtFilter authenticates the request

```
String token = extractFromHeader();
jwtUtil.validate(token);

UserDetails user = userDetailsService.loadUserByUsername(username);

Authentication auth =
    new UsernamePasswordAuthenticationToken(
        user,
        null,
        user.getAuthorities()
    );

SecurityContextHolder.getContext().setAuthentication(auth);
```

✅ **Authentication is now stored globally for this request**

3️⃣ Spring checks authorization

Before controller execution:

```
FilterSecurityInterceptor
```

It checks:

- Is `SecurityContextHolder.getAuthentication()` present?
- Is it authenticated?
- Are roles allowed?

If ❌→ 401 / 403
If ✅→ controller allowed

4️⃣ Controller method is invoked

Now Spring MVC runs:

```
public TodoEntity create(@RequestBody TodoEntity todo,
                         Authentication auth)
```

❓ Where does `auth` come from?

♆ Spring MVC sees:

> "This method wants an `Authentication`"

So it internally does:

```
Authentication auth =
    SecurityContextHolder.getContext().getAuthentication();
```

And injects it.

🚫 **No provider**
🚫 **No re-authentication**
🚫 **No boolean return**

🧰 Why AuthenticationProvider is NOT involved here

AuthenticationProvider is only used when:

```
authenticationManager.authenticate(...)
```

That happens:

- ✔️Login
- ✔️Username/password
- ❌Not JWT requests

JWT requests are already authenticated.

🛑Why Spring trusts JwtFilter

Because:

1. JwtFilter is in **SecurityFilterChain**
2. It runs **before controllers**
3. It sets Authentication
4. Spring trusts the SecurityContext

   Spring Security works on **trust boundaries**
   Filters are trusted, controllers are not

🚫Why this counts as “authenticated”

Because this line:

```
new UsernamePasswordAuthenticationToken(user, null, authorities);
```

Creates an object where:

```
isAuthenticated() == true
```

So Spring treats it as authenticated.

Mental Model (IMPORTANT)

Think of Spring Security like airport security:

StageResponsibility	
Filter	Identity verification
SecurityContext	Passport storage
Authorization	Check permission

Stage	Responsibility
Controller	Business logic

Controllers **never verify passports**
They only **read them**

🛄 Proof (You can test this)

Add this inside controller:

```
System.out.println(auth.isAuthenticated()); // true
System.out.println(auth.getPrincipal());    // UserDetails
```

🆎 Final Summary

Question	Answer
How does Authentication arrive?	From SecurityContextHolder
Who sets it?	JwtFilter
Does controller authenticate?	❌ No
Does AuthenticationProvider run?	❌ No
Why is request allowed?	Auth already exists
Who decides true/false?	Filters + authorization phase

You now understand **Spring Security better than 90% of devs**.

If you want next:

- Write a **custom argument resolver**
- Trace Spring source code
- Build **stateless session debugging**
- Add role-based guards

Just say the word ☑️

9️⃣How to define your own provider (optional but powerful) @Component public class MyAuthProvider implements AuthenticationProvider { @Override public Authentication authenticate(Authentication auth) { // custom logic } @Override public boolean supports(Class\<?> clazz) { return MyAuthToken.class.isAssignableFrom(clazz); } }Make it for me Spring auto-picks it ☑️make it for me

Perfect. Let's **actually build a real custom AuthenticationProvider** so you see **Spring auto-picks it**, no magic left.

I'll do this in **small, solid steps** ⚫

🔝 Goal

Create:

- ✅ Custom `AuthenticationToken`
- ✅ Custom `AuthenticationProvider`
- ✅ Use `AuthenticationManager.authenticate()`
- ✅ Spring auto-selects provider via `supports()`

1️⃣ Create a Custom Authentication Token

This represents **WHAT kind of authentication you want**.

`MyAuthToken.java`

```
import
org.springframework.security.authentication.AbstractAuthenticationToken;
import org.springframework.security.core.GrantedAuthority;

import java.util.Collection;

public class MyAuthToken extends AbstractAuthenticationToken {

    private final Object principal;
    private Object credentials;

    // BEFORE authentication
    public MyAuthToken(String username, String secret) {
        super(null);
        this.principal = username;
        this.credentials = secret;
        setAuthenticated(false);
    }

    // AFTER authentication
    public MyAuthToken(Object principal,
                       Collection<? extends GrantedAuthority> authorities) {
        super(authorities);
        this.principal = principal;
        this.credentials = null;
        setAuthenticated(true);
```

```
    }

    @Override
    public Object getCredentials() {
        return credentials;
    }

    @Override
    public Object getPrincipal() {
        return principal;
    }
}
```

⚫This is **exactly how `` works internally**.

2️⃣Create Custom AuthenticationProvider

`MyAuthProvider.java`

```
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.stereotype.Component;

import java.util.List;

@Component
public class MyAuthProvider implements AuthenticationProvider {

    @Override
    public Authentication authenticate(Authentication authentication) {

        String username = authentication.getPrincipal().toString();
        String secret = authentication.getCredentials().toString();

        // ☑️ CUSTOM LOGIC
        if (!"admin".equals(username) || !"secret123".equals(secret)) {
            throw new RuntimeException("Invalid credentials");
        }

        // AUTHENTICATED TOKEN
        return new MyAuthToken(
                username,
                List.of(new SimpleGrantedAuthority("ROLE_USER"))
        );
    }

    @Override
```

```
    public boolean supports(Class<?> authType) {
        return MyAuthToken.class.isAssignableFrom(authType);
    }
}
```

✅**THIS is where Spring decides to use your provider**

3️⃣Use It in Controller (No wiring needed!)

`AuthController.java`

```
@RestController
@RequestMapping("/auth")
public class AuthController {

    private final AuthenticationManager authenticationManager;

    public AuthController(AuthenticationManager authenticationManager) {
        this.authenticationManager = authenticationManager;
    }

    @PostMapping("/login")
    public String login(@RequestBody LoginRequest req) {

        Authentication auth = authenticationManager.authenticate(
                new MyAuthToken(req.getUsername(), req.getSecret())
        );

        return "Authenticated as: " + auth.getPrincipal();
    }
}
```

🚦**Notice**

- You did NOT register provider
- You did NOT select provider
- Spring did everything

4️⃣How Spring Auto-Picks Your Provider ✅

Internally, Spring does:

```
for (AuthenticationProvider provider : providers) {
    if (provider.supports(authentication.getClass())) {
        return provider.authenticate(authentication);
```

```
    }
}
```

So when you pass:

```
new MyAuthToken(...)
```

Spring asks:

> "Which provider supports this token?"

✔️ `MyAuthProvider.supports(MyAuthToken.class)` → TRUE
❌ All others → FALSE

➡️ **Your provider is selected**

5️⃣ Why This Is Powerful

You can now authenticate with:

- API keys
- OTP
- Fingerprints
- Hardware tokens
- External systems
- Custom headers

WITHOUT touching Spring internals.

6️⃣ Compare with JWT

Scenario	Provider Used
Login with password	DaoAuthenticationProvider
Custom auth	MyAuthProvider
JWT request	❌ No provider
OAuth2	OAuth2AuthenticationProvider

7️⃣ Golden Rule (MEMORIZE)

> **AuthenticationManager routes**

> **AuthenticationProvider validates**
>
> **AuthenticationToken defines the contract**

8️⃣ You Are Now Advanced-Level

You now understand:

- Filters
- AuthenticationManager
- Providers
- Tokens
- JWT vs credentials
- Why Spring feels “automatic”

If you want next:

- 🔐 JWT AuthenticationProvider (advanced)
- 🧩 Multiple providers + fallback
- 🧭 Spring Security source walk-through
- 🔒 OAuth2 internals

Just tell me ✅

public MyAuthToken(Object principal, Collection\<? extends GrantedAuthority> authorities) { super(authorities); this.principal = principal; this.credentials = null; setAuthenticated(true); } How does ti work how does it know which method to call when and where before authentication and after authentication

Excellent question — this is the **core of Spring Security internals**, and once this clicks, everything becomes clear.
I’ll explain this **step-by-step, timeline-based**, and tie it **directly to your constructor**.

⚫ First, one golden rule (very important)

> **Spring Security NEVER calls constructors magically.**
> **YOU call constructors.**
> **Spring only *routes* Authentication objects through filters & providers.**

So the question is not *“how does Spring know which constructor to call?”*
The real question is:

> **How does Spring know WHEN this token represents “before authentication” vs “after authentication”?**

Answer:
♓ ``** flag + which constructor you used**

Core interfaces involved

```
Authentication        // interface
AbstractAuthenticationToken
MyAuthToken           // your class
AuthenticationProvider
AuthenticationManager
SecurityContext
```

🏗️ Your constructor (AFTER authentication)

```
public MyAuthToken(Object principal,
                   Collection<? extends GrantedAuthority> authorities) {
    super(authorities);
    this.principal = principal;
    this.credentials = null;
    setAuthenticated(true);
}
```

This constructor means:

✅ User is **ALREADY authenticated**
✅ Authorities are **known**
✅ No credentials needed anymore
✅ Token is **trusted**

❗ But what about BEFORE authentication?

You MUST have another constructor:

```
public MyAuthToken(Object principal, Object credentials) {
    super(null);
    this.principal = principal;
    this.credentials = credentials;
    setAuthenticated(false);
}
```

This token means:

❌ Not authenticated
❌ Just username + password
❌ Needs verification

🔁Full Authentication FLOW (REAL LIFE)

Let's walk through **exact execution order**.

🟢FLOW 1: LOGIN REQUEST (`/login`)

Step 1️⃣Controller or Filter creates token (YOU do this)

```
Authentication authRequest =
    new MyAuthToken(username, password); // unauthenticated
```

At this moment:

```
isAuthenticated = false
```

Step 2️⃣AuthenticationManager routes it

```
authenticationManager.authenticate(authRequest);
```

Internally Spring does:

```
for (AuthenticationProvider provider : providers) {
    if (provider.supports(authRequest.getClass())) {
        return provider.authenticate(authRequest);
    }
}
```

✅THIS is where **supports()** matters.

Step 3️⃣Your Provider is selected automatically

```
@Override
public boolean supports(Class<?> clazz) {
    return MyAuthToken.class.isAssignableFrom(clazz);
}
```

✔️Spring sees `MyAuthToken`
✔️Picks `MyAuthProvider`

Step 4️⃣ authenticate() is called

```
@Override
public Authentication authenticate(Authentication auth) {

    String username = auth.getName();
    String password = auth.getCredentials().toString();

    // validate from DB
    User user = userService.loadUser(username);

    if (!passwordEncoder.matches(password, user.getPassword())) {
        throw new BadCredentialsException("Invalid credentials");
    }

    // ✅ CREATE NEW TOKEN (IMPORTANT)
    return new MyAuthToken(user, user.getAuthorities());
}
```

🚨 CRITICAL POINT:

> **You DO NOT modify the old token**
> **You CREATE A NEW TOKEN**

Old token ❌ → unauthenticated
New token ✅ → authenticated

Step 5️⃣ AuthenticationManager returns AUTHENTICATED token

```
Authentication authenticatedToken = provider.authenticate(...)
```

Now:

```
isAuthenticated = true
principal = User
authorities = ROLE_USER
```

Step 6️⃣ Stored in SecurityContext

```
SecurityContextHolder.getContext()
    .setAuthentication(authenticatedToken);
```

This is the **single source of truth**.

🟢FLOW 2: JWT REQUEST (NO LOGIN)

Now the most important part — your confusion area ⚫

Step 1️⃣JWT Filter runs (Before Controller)

```
String token = extractJwt(request);
String username = jwtUtil.extractUsername(token);
```

Step 2️⃣JWT is validated

```
if (jwtUtil.isValid(token)) {
```

Step 3️⃣YOU create AUTHENTICATED token directly

```
User user = userService.loadUser(username);

Authentication auth =
    new MyAuthToken(user, user.getAuthorities());

SecurityContextHolder.getContext().setAuthentication(auth);
```

🚾Notice:

❌No AuthenticationManager
❌No Provider
❌No password
❌No supports()

Why allowed?

♓**JWT itself is the proof**

Why Spring does NOT call AuthenticationProvider here?

Because:

**AuthenticationProvider is ONLY used when you call **``

JWT filter **does not** call it.

🟢FLOW 3: Controller receives Authentication

```
@PostMapping("/todos")
public TodoEntity create(@RequestBody TodoEntity todo,
                         Authentication auth) {
```

How does `auth` arrive here?

Step 1️⃣Spring MVC argument resolver

Spring sees:

```
Authentication auth
```

Internally:

```
SecurityContext context =
    SecurityContextHolder.getContext();

Authentication auth = context.getAuthentication();
```

✔️Already authenticated
✔️Already trusted
✔️No provider needed

❓Why no true/false return?

Because:

```
isAuthenticated == true
```

That is the **boolean proof**.

☑️Key Insight (MOST IMPORTANT)

ScenarioWho creates tokenAuthenticated?Provider used?			
Login	Controller/Filter	❌→ ✅	YES
JWT	Filter	✅	NO
Controller	Spring	already ✅	NO

Why constructor matters so much

ConstructorMeaning	
`(principal, credentials)`	BEFORE auth
`(principal, authorities)`	AFTER auth

Spring **never guesses** — **YOU decide**.

🚆Final mental model

> **Authentication is just a DATA OBJECT**
> Spring only **moves it around**
> YOU decide when it becomes trusted

If you want, next I can:

- Draw **full ASCII flow diagram**
- Show **exact Spring source code**
- Explain **why setAuthenticated(true) is protected**
- Compare with `UsernamePasswordAuthenticationToken`

Just tell me ☑️

Authentication auth = new MyAuthToken(user, user.getAuthorities()); how does it know here which constructor to called cause when we send password it called before authentication now how its calling after authentication constructore

This is a **very sharp Java + Spring Security question**, and the answer is **pure Java**, not Spring magic.

Let's clear the confusion **once and for all**.

⚫Short answer (core truth)

> **Spring does NOT decide which constructor to call.**
> **JAVA decides at COMPILE TIME based on the arguments you pass.**

That's it.
Everything else is layering.

Why this feels confusing

You see this:

```
new MyAuthToken(user, user.getAuthorities());
```

and earlier:

```
new MyAuthToken(username, password);
```

and you think:

> "Spring knows this is after authentication"

❌**No**

♎**YOU are explicitly choosing the constructor**

🗄️Let's look at your constructors side-by-side

```
// BEFORE authentication
public MyAuthToken(Object principal, Object credentials) {
    super(null);
    this.principal = principal;
    this.credentials = credentials;
    setAuthenticated(false);
}
```

```
// AFTER authentication
public MyAuthToken(Object principal,
                   Collection<? extends GrantedAuthority> authorities) {
    super(authorities);
    this.principal = principal;
    this.credentials = null;
```

```
    setAuthenticated(true);
}
```

⚠️What Java does internally

Case 1️⃣

```
new MyAuthToken(username, password);
```

Types:

```
(Object, Object)
```

✔️Matches constructor #1
✔️Java chooses it **at compile time**

Case 2️⃣

```
new MyAuthToken(user, user.getAuthorities());
```

Types:

```
(Object, Collection<GrantedAuthority>)
```

✔️Matches constructor #2
✔️Java chooses it **at compile time**

❗ Spring is NOT involved here

There is **no reflection**, **no runtime decision**, **no Spring logic**.

This is **standard Java constructor overloading**.

Then how does Spring “understand” authentication state?

Answer: **it doesn’t think — it checks flags**

```
auth.isAuthenticated()
```

That's it.

🚫 Why `setAuthenticated(true)` matters

Spring Security has this rule:

```
if (auth.isAuthenticated()) {
    // TRUST this authentication
}
```

So when you do:

```
new MyAuthToken(user, authorities);
```

you are saying:

> "I guarantee this user is authenticated."

🚦 Important security rule (very important)

Spring enforces this:

```
@Override
public void setAuthenticated(boolean isAuthenticated) {
    if (isAuthenticated) {
        throw new IllegalArgumentException(
            "Cannot set this token to trusted without authorities"
        );
    }
}
```

That's why:

✅ After-auth constructor has authorities
❌ Before-auth does not

---

4 Real mental model (remember this)

> **Constructors define the SECURITY STATE of the token**

Constructor	Meaning
`(principal, credentials)`	“Please authenticate this user”
`(principal, authorities)`	“User is already authenticated”

Timeline visualization

```
LOGIN REQUEST
    ↓
new MyAuthToken(username, password)
    ↓
AuthenticationManager
    ↓
MyAuthProvider
    ↓
new MyAuthToken(user, authorities)
    ↓
SecurityContextHolder
```

```
JWT REQUEST
    ↓
JWT validated
    ↓
new MyAuthToken(user, authorities)
    ↓
SecurityContextHolder
```

✅ Final key sentence (memorize this)

> **Spring does not choose constructors.**
> **Java chooses constructors.**
> **Spring only trusts what you hand it.**

If you want, next I can:

- Show Spring’s **actual source code**
- Explain why `UsernamePasswordAuthenticationToken` is designed this way
- Show what happens if you accidentally call the wrong constructor (security hole)

Just say ♀️

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

Case 2: JWT Authentication

YOU manually create:

```
new UsernamePasswordAuthenticationToken(
    userDetails,
    null,
    userDetails.getAuthorities()
)
```

Because:

- Credentials already validated by JWT
- No DB password check needed

How AuthenticationManager Chooses AuthenticationProvider

1. AuthenticationManager holds **list of providers**
2. Calls `supports(authentication.getClass())`
3. First provider that returns `true` is used

DaoAuthenticationProvider supports:

```
UsernamePasswordAuthenticationToken
```

So when login request comes:

- Filter creates UsernamePasswordAuthenticationToken
- Manager routes it to DAO provider

✔️That's how it "knows"

⚠️Why JWT Uses Filter, NOT AuthenticationProvider

Login	JWT Request
Credentials	Token
Needs validation	Already validated
Uses AuthenticationManager	Uses Filter
Uses Provider	No Provider

JWT = **stateless proof**, not authentication process

🧳Where Token Is VALIDATED

Inside `JwtUtil`:

- Signature verification
- Expiry check
- Subject (username) check

Spring Security does **NOT** validate JWT for you.

YOU do it before setting Authentication.

Deep Dive: Spring Security Filters

Spring Security = **Filter Chain**

```
Request → Filter1 → Filter2 → ... → Controller
```

Each filter:

```
doFilter(request, response, chain)
```

- Before `chain.doFilter()` → pre-processing
- After `chain.doFilter()` → post-processing

Important Filters

- SecurityContextPersistenceFilter
- UsernamePasswordAuthenticationFilter
- JwtFilter (custom)
- AuthorizationFilter

Filters are **middleware**.

🚫CSRF vs JWT (Why CSRF Is Disabled)

CSRF Protects:

- Session-based auth
- Cookies automatically sent

JWT:

- Token sent manually in header
- No automatic browser behavior

❌CSRF unnecessary

So we disable:

```
http.csrf().disable();
```

✔️Safe for JWT

🗂️Complete JWT Todo API (Structure)

```
config/
 └─ SecurityConfig.java   ← AuthenticationManager here
filter/
 └─ JwtFilter.java        ← JWT validation here
util/
 └─ JwtUtil.java          ← token validation logic
service/
 ├─ CustomUserDetailsService.java
 └─ TodoService.java
controller/
 ├─ AuthController.java   ← login
 └─ TodoController.java   ← secured endpoints
```

Final Mental Model

- Filters = request pipeline
- AuthenticationManager = router
- AuthenticationProvider = credential validator
- JWT = stateless identity proof
- SecurityContext = logged-in user storage

Once you understand this, Spring Security becomes **predictable**, not magic.

🚫What You Should Master Next

1️⃣Write JWT filter from scratch 2️⃣Log filter execution order 3️⃣Create custom AuthenticationProvider 4️⃣Role-based authorization 5️⃣Stateless vs Stateful auth

---

🔁 Request-by-Request Execution Flow (JWT Secured API)

```
Client
│ Authorization: Bearer <JWT>
▼
JwtFilter (OncePerRequestFilter)
│ extract token
│ validate token (signature + expiry)
│ loadUserByUsername()
│ create UsernamePasswordAuthenticationToken (AUTHENTICATED)
│ SecurityContextHolder.setAuthentication()
▼
Spring Security Authorization
│ checks authentication + roles
▼
Controller (@RestController)
```

What Happens Line-by-Line (Very Important)

1️⃣ **Client sends request** with JWT in header

2️⃣ **JwtFilter runs BEFORE controller**

- Spring auto-runs filters via `FilterChainProxy`
- You did NOT call this filter manually

3️⃣ **Token Validation Happens Here**

- Signature check
- Expiry check
- Username extraction

4️⃣ **UserDetails Loaded**

- From DB / memory

5️⃣ **Authentication Object Created**

```
UsernamePasswordAuthenticationToken auth =
    new UsernamePasswordAuthenticationToken(
        userDetails, null, userDetails.getAuthorities());
```

✔️ This means: **USER IS AUTHENTICATED**

6️⃣ **Authentication Stored**

```
SecurityContextHolder.getContext().setAuthentication(auth);
```

7️⃣ Authorization Phase

- Spring checks `isAuthenticated()`
- Checks roles if configured

8️⃣ Controller Executes

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String username = auth.getName();
```

🕎 FULL JWT TODO API (SPRING BOOT)

☢️ Package Structure

```
com.example.todo
├── config
│   └── SecurityConfig.java
├── controller
│   ├── AuthController.java
│   └── TodoController.java
├── filter
│   └── JwtFilter.java
├── model
│   ├── UserEntity.java
│   └── Todo.java
├── repository
│   ├── UserRepository.java
│   └── TodoRepository.java
├── security
│   └── CustomUserDetailsService.java
├── util
│   └── JwtUtil.java
```

🔞 SecurityConfig (AuthenticationManager USED HERE)

```
@Bean
public AuthenticationManager
authenticationManager(AuthenticationConfiguration config)
        throws Exception {
    return config.getAuthenticationManager();
}
```

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.csrf().disable()
        .authorizeHttpRequests()
        .requestMatchers("/auth/**").permitAll()
        .anyRequest().authenticated()
        .and()
        .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```

✔️AuthenticationManager is **AUTO-BUILT**✔️ Uses `DaoAuthenticationProvider`

⚫AuthController (AuthenticationManager IN ACTION)

```
@PostMapping("/login")
public String login(@RequestBody LoginRequest req) {
    Authentication auth = authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(req.getUsername(),
req.getPassword())
    );

    return jwtUtil.generateToken(auth.getName());
}
```

✔️ **Authentication object created by YOU**✔️ AuthenticationManager sends it to Provider

---

🗂️JwtFilter (NO AuthenticationManager HERE)

```
protected void doFilterInternal(...) {
    String token = extractToken(request);

    if (jwtUtil.validate(token)) {
        String username = jwtUtil.extractUsername(token);

        UserDetails user = userDetailsService.loadUserByUsername(username);

        UsernamePasswordAuthenticationToken auth =
            new UsernamePasswordAuthenticationToken(user, null,
user.getAuthorities());

        SecurityContextHolder.getContext().setAuthentication(auth);
    }
```

```
    chain.doFilter(request, response);
}
```

✔️JWT does **NOT** call AuthenticationManager ✔️JWT is already authenticated

---

AuthenticationManager vs Provider (CLEAR)

AuthenticationManager

- Coordinator
- Delegates to providers

DaoAuthenticationProvider

- Used for username/password login
- Uses:
- UserDetailsService
- PasswordEncoder

Why JWT Has NO Provider?

Login	JWT Request
Needs password	Already trusted
Uses provider	Uses filter

❓How Authentication Object Exists Without Us Creating It

Situation	Who Creates It
Login	Controller (you)
JWT	JwtFilter
OAuth	OAuth Filter
Session	Session Filter

✔️Spring Security reads **SecurityContextHolder** only

🚻CSRF vs JWT (WHY DISABLED)

CSRF

- Protects cookie-based sessions
- Browser auto-sends cookies

JWT

- Token sent manually in header
- No browser auto-send

❌CSRF useless for JWT

```
http.csrf().disable();
```

FINAL MENTAL MODEL

> "Spring Security does NOT authenticate users. Filters and Providers do. Spring only ENFORCES the result."

🚷What You Should Master Next

1️⃣ Write your own AuthenticationProvider 2️⃣Log filter chain execution order 3️⃣Add role-based authorization 4️⃣Build session-based auth 5️⃣Break JWT intentionally and debug it